AuxFlow: A Static Applicability Test for Fixpoint Convergence in Multi-Pass LaTeX Builds

Upayan Mazumder
School of Computer Science and Engineering (SCOPE)
Vellore Institute of Technology
Vellore, Tamil Nadu, India
upayan.mazumder2024@vitstudent.ac.in

Banupriya R
School of Computer Science and Engineering (SCOPE)
Vellore Institute of Technology
Vellore, Tamil Nadu, India
banupriya.r@vit.ac.in

***Abstract*—A LaTeX document with cross-references, a table of contents, or a bibliography needs several engine runs. Each run reads the auxiliary files that the previous run wrote. Five widely used tools decide when to stop with five different tests, and four of them also cap the run count with a fixed constant. None of them derives the run count from the source. This paper describes AuxFlow, a compiler front end and static analyzer for a declared subset of LaTeX. AuxFlow lowers a document to a graph of auxiliary fact instances with lagged edges. It finds the facts whose values reach a construct that controls how much content the document generates. It then classifies the document before any engine run. We test AuxFlow on 15 documents, seven stopping strategies, and ten repetitions in a pinned container (1050 records). All 7 documents that AuxFlow certifies receive a bound that is at least the measured run count. AuxFlow flags all 6 non-convergent documents before it starts the engine. Its analysis takes 1.3 ms at the median and 10.4 ms at most. We also report a guard that we added after the first measurement. The guard changed three declared false negatives into UNKNOWN results.**

***Index Terms*—LaTeX, build systems, static analysis, fixpoint iteration, attribute grammars**

I. Introduction

LaTeX resolves a cross-reference in two steps. The first engine run writes the value of each label to an auxiliary file. The next engine run reads that file and prints the value. A table of contents, a bibliography, and a list of figures work the same way. The number of runs that a document needs is therefore a property of its reference structure.

In practice, tools discover this number by running the engine. `latexmk` compares checksums of the files that the engine wrote and stops after five repeats by default [1]. Tectonic stops after six passes [2]. ConTeXt stops after nine. Typst stops after five layout iterations [3]. Table I lists the five tests and their limits. No tool derives a limit from the document.

Some documents never settle. Gaspar gave a one-page document whose page reference grows by one on every run [4]. A loop bounded by the reference emits one more page each time. When a tool reaches its limit, three of the five tools report no cause.

We ask a narrower question than "how many runs does this document need?". Prior work answers the general question for fixed graphs. Static pass numbering is Bochmann's [5] and Alblas's [6]. Bounding fixpoint rounds by a graph parameter is Hecht and Ullman's [7]. Splitting a circular dependency graph into strongly connected components is Jones's [8]. Our setting differs in one respect. A document can change its own dependency graph. A page reference can change the page count, and the page count decides how many page references exist. The classical theory does not cover that case.

AuxFlow tests whether the classical theory applies to a given document. The test is conservative and covers a declared subset of LaTeX. It is a sufficient condition for the finite-height half of the convergence condition of Magnusson and Hedin [9]. It does not test the monotonicity half.

This paper makes three contributions.

1) A declared LaTeX subset, an instance-level graph of auxiliary facts with pass-lag edges, and an influence analysis that classifies a document before it runs (Section III).
2) A measured comparison of five stopping tests and two AuxFlow configurations on a fifteen-document corpus. The corpus covers nine published non-convergence mechanisms (Section V).
3) A report of what the analysis cannot see, including a change that we made after the first measurement (Section VI).

We do not claim to have invented static pass numbering, strongly connected component analysis, graph-derived round bounds, or checksum verification. Section II places each of these in earlier work.

II. Background and Related Work

A. The auxiliary round trip

A `\label` command writes a `\newlabel` record to the `.aux` file. At `\begin {document}`, the kernel reads the

TABLE I
STOPPING TESTS AND LIMITS IN FIVE DEPLOYED TOOLS. VALUES COME FROM MANUALS AND SOURCE CODE (LATEXMK 4.88).

Tool	Test between runs	Limit	Cause on limit
latexmk	time, size, and MD5 of all files	5	none
latexrun [10]	tool inputs unchanged	none[a]	none
Tectonic	file read, then rewritten, in one run	6	file name
ConTeXt	MD5 of the `.tuc` file	9	none
Typst	hash of layout iterations	5	spans and value history

[a]No limit is documented. Our run stopped after 10 iterations (Section V-D).

`.aux` file from the previous run before it opens the file for writing. A value written in run p is therefore readable first in run $p+1$. We call this delay the *pass lag*, and it is exactly one. Bibliography data pass through BibTeX between two engine runs.

B. Multi-pass analysis and circular attributes

Bochmann decides how many left-to-right passes an attribute grammar needs [5]. Alblas separates *simple* multi-pass grammars from *pure* ones. In the pure case every instance of an attribute can take its own pass number [6]. Deciding the pure case is hard, and finding the minimum number of passes is NP-complete [11]. AuxFlow numbers fact instances, so it works in Alblas's pure setting. It computes a conservative bound and does not claim a minimum.

Farrow gives conditions under which a circular grammar has a well-defined result [12]. Jones evaluates such grammars by splitting the dependency graph into strongly connected components and ordering them topologically [8]. Magnusson and Hedin state the convergence condition: the attribute values on a cycle must form a lattice of finite height, and the functions must be monotone. They also state that their evaluator does not check it [9]. AuxFlow supplies a domain-specific check for the first half of that condition.

Krishnan and Van Wyk analyze termination for higher-order attribute grammars, where evaluation creates new trees [13], [14]. Their analysis works on a fixed grammar and uses a term-ordering proof. AuxFlow works on the fact graph of one document and uses reachability. We have not read the work of Söderberg and Hedin on circular higher-order attributes in full [15]. We therefore make no claim against it.

C. Bounds on fixpoint rounds, rates, and build systems

Hecht and Ullman bound the rounds of an iterative data-flow solver by the depth of the flow graph [7], [16]. The AuxFlow plan has the same shape: P runs to propagate and one run to confirm. Lee and Messerschmitt separate nodes with a fixed output rate from nodes whose rate depends on data [17]. AuxFlow draws the same line for documents. A value that changes only *what* the document says is benign. A value that changes *how much* content the document generates is not.

Mokhov et al. separate dependencies that are known statically from dependencies that depend on computed values [18]. Erdweg et al. build LaTeX with a fixpoint cycle handler and conclude that "soundly detecting dependencies is a lost cause" for LaTeX, because even parsing is undecidable [19], [20]. Our answer is the declared subset. Any construct outside it becomes an opaque fact, and an opaque fact makes the document UNKNOWN.

Mädje formalizes document stability as a recurrence, proves it undecidable by Rice's theorem for arbitrary code, and proposes a fixed iteration cap with an error at the guilty call [21]. AuxFlow follows the same framing and adds a static decision for a restricted class.

D. Non-convergence mechanisms

Published sources describe at least nine mechanisms. Table II lists the ones that we model as corpus members. M1 and M2 are loops and conditionals that depend on a page reference. M3 to M7 change the typeset extent of a reference, a float, or a note. M8 is a nondeterministic auxiliary record. M9 is a false positive of the kernel warning and is not real non-convergence.

III. APPROACH

Figure 1 shows the pipeline. Everything up to the build plan runs without a TeX engine. Only `auxflow run` starts a subprocess.

A. Declared subset and opaque facts

The front end has a category-code lexer, a recursive-descent parser, a multi-file syntax tree, and symbol tables for labels, references, citations, packages, files, and counters. Every node carries a source span. The subset covers sectioning, labels, references, citations, captions, generated lists, counters, and loops and conditionals over counters. A small list of packages and commands is inert, because they write no auxiliary state. Any other command or package becomes an opaque fact. We do not expand macro bodies. A local macro is inert only if every command in its body is inert.

B. The Auxiliary Reference Graph

The Auxiliary Reference Graph (ARG) has one node for each fact *instance*. An earlier design used one node for each fact *kind*. That graph is the same eight-node lattice for every document, so analyzing it is a table lookup. Instance nodes make the problem a real data-flow problem.

Figure 2 shows the data model. A fact has a kind (label number, label page, reference use, contents entry, list extent, citation use, bibliography body, counter read, content volume, or opaque), a key, a component, and a source span. The component is either `exists` or `value`. A dependency edge has a lag and a type. The types are AUX_ROUNDTRIP,

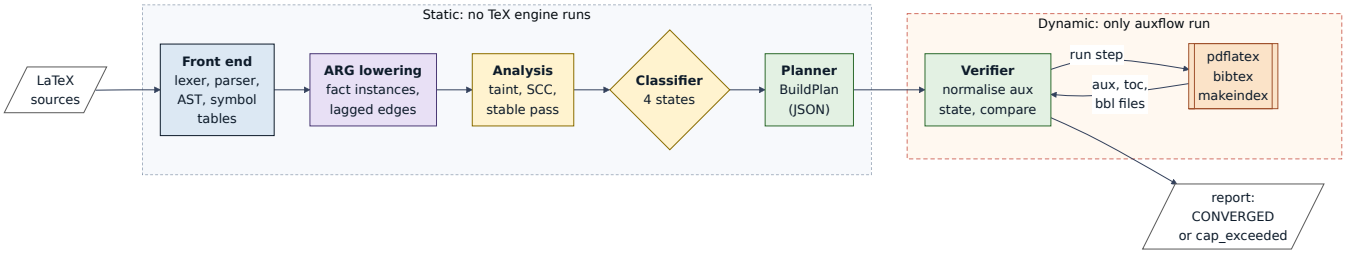


Fig. 1. The AuxFlow pipeline. The static stages never start a TeX engine. The verifier is the only stage that does.

TOOL, LAYOUT, and STRUCTURAL. Every edge carries a reason string. The reason is the text that the diagnostics print.

C. Stable-pass numbering

For a fact f with dependencies $\mathrm{deps}(f)$, the stable pass is

$$s(f) = \begin{cases} 1 & \mathrm{deps}(f) = \varnothing, \\ 1 + \max\limits_{g \in \mathrm{deps}(f)} \big(s(g) + \ell(g \to f) - 1\big) & \text{otherwise}, \end{cases} \tag{1}$$

where ℓ is the lag of the edge. Let P be the largest $s(f)$ over the facts that the engine stores in auxiliary files. The predicted run count is $N = P + 1$. A tool edge counts as lag zero in Eq. (1), because BibTeX runs between two engine runs. The numbering follows the longest path on the condensation graph. It is a bound and not a minimum.

D. Influence analysis

The ARG joins every generated list to a synthetic CONTENT_VOLUME fact that stands for pagination. The `exists/value` split removes the edge from a fact's value to pagination. Without that split, every document with one `\ref` would form a single component, and the analysis would say nothing.

Taint analysis restores the one dependency that matters. A seed is a fact whose value is read in the bound or condition of a control construct. A worklist propagates the taint along value edges. For each tainted fact, the analysis adds a STRUCTURAL edge with lag one to CONTENT_VOLUME. The edge carries the source location of the read. Each fact enters the worklist at most once, so the cost is $O(V + E)$.

An iterative Tarjan algorithm then finds the strongly connected components in $O(V + E)$. It uses an explicit stack, because a recursive version fails on the larger corpus member. A non-trivial component has one of three labels. It is AMPLIFYING if it contains an internal structural edge. It is UNKNOWN_CYCLE if it contains an opaque fact. Otherwise it is BENIGN.

The analysis over-approximates. Taint follows any value edge and does not check that the influence changes a quantity. A benign document can therefore look amplifying, and the user pays with one flag. The analysis cannot make the opposite mistake on modeled constructs. It can, however, miss any feedback that has no seed.

E. Classification and planning

Figure 3 shows the decision. The precedence is AMPLIFYING, then UNKNOWN, then BENIGN_CYCLIC, then CERTIFIED. An amplifying cycle wins over an opaque fact, because it gives the user a more useful report.

The planner enforces one safety rule with an assertion. Only a certified plan uses the derived count. Every other executable plan has at least three engine runs. An amplifying document gets no plan unless the user passes `--force`. Then the plan has three runs and a cap of six. The planner also removes work. It scans only files that the entry file reaches. It runs BibTeX only if the document has both a citation and a bibliography command.

F. Runtime verification

Engine warnings are not a sound test. The kernel warns when a label changed, but base LaTeX gives no warning when a table of contents changed [23]. Active characters in captions cause false warnings [24]. The AuxFlow verifier therefore reads the state directly.

After each engine run, the verifier parses the `.aux` files, the generated list files, and the bibliography body into a set of canonical records. It discards order, white space, and the argument-free `\relax` command. It keeps every unrecognized record, so a changing unknown record cannot look stable. It compares the set with the set from the previous run. Figure 4 shows the loop. If the plan ends and the state still changes, the verifier repeats the last engine step until the state is stable or the cap is reached. It records a *prediction miss* whenever the observed run count exceeds a certified bound.

Each tool call has a time limit of 600 seconds. The verifier passes `-no-shell-escape` to `pdflatex`. It rejects auxiliary files that resolve outside the work directory. AuxFlow does not sandbox TeX, so a user must run only trusted documents.

The verifier is a reimplementation of the checksum approach of `latexmk`. We do not claim it as a contribution. We reimplemented it so that baselines B, C, and E share one normalization and differ only in the stopping test.

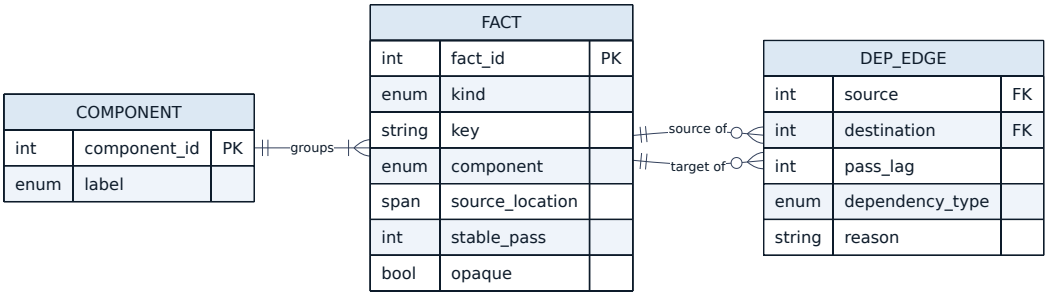


Fig. 2. Data model of the ARG. A document lowers to a set of FACT and DEP_EDGE records. Tarjan's algorithm [22] groups facts into components. Each fact keeps its source span, so every diagnostic has a `file:line:column` location.

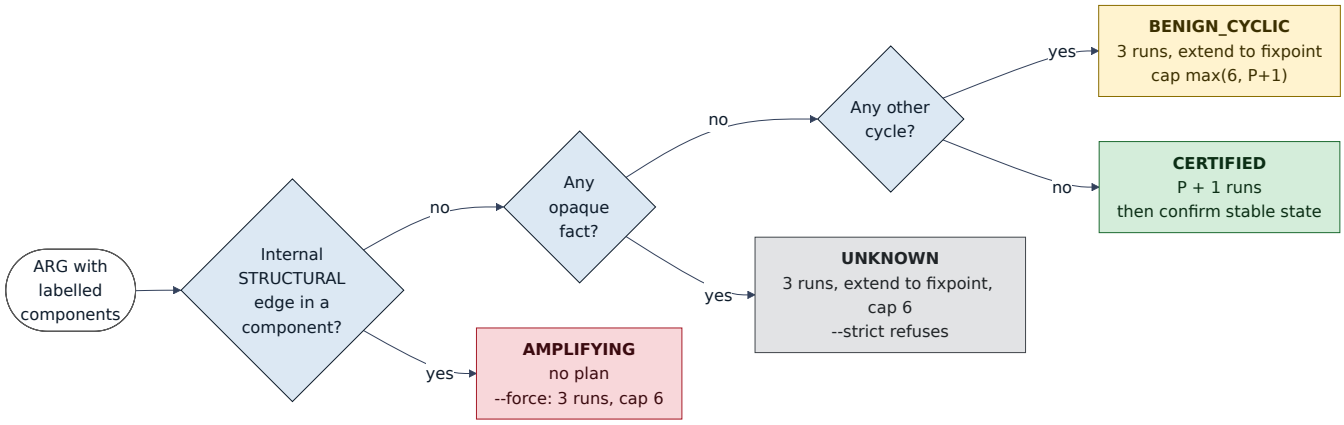


Fig. 3. Classification and the plan that each state receives. P is the largest stable pass over stored facts. The plan for a non-certified document never has fewer than three engine runs.

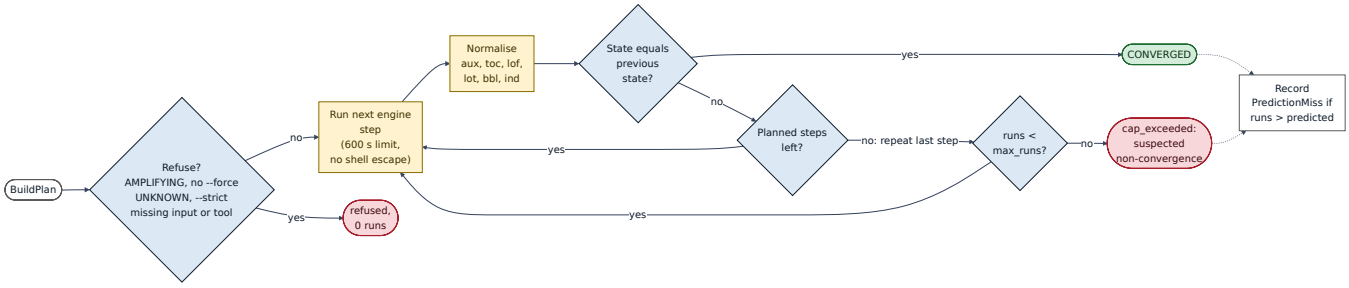


Fig. 4. The verification loop of `auxflow run`. A plan that ran to its last step is checked, and extended only while the auxiliary state still changes.

IV. Worked Example

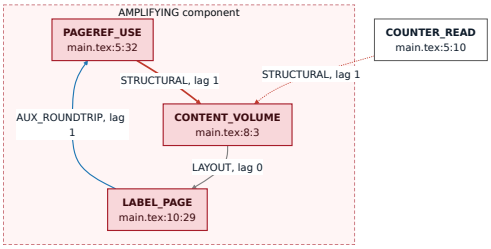
Figure 5 shows document `b08` of our corpus. It is Gaspar's mechanism M1 in a compact form.

```
5 \whiledo{\value{generatedpage}
6          <\pageref{end:generated}}{%
7   \stepcounter{generatedpage}%
8   Record sheet occupies its own page.
9   \newpage }
10 End of the record.\label{end:generated}
```

The loop reads the page of the label. Each pass of the loop emits one page. The label sits after the loop, so the loop changes the page that it reads.

The taint analysis seeds the page reference at line 5, because the loop condition reads it. The analysis adds a structural edge to the volume of content. The layout edge then reaches the label page, and the round trip returns to the reference. The three facts form one component with an internal structural edge, so the document is amplifying. AuxFlow reports this without an engine run. The report names the chain and the three source locations.

With `--force`, AuxFlow runs the document under the cap. The page count grows by one on every run,




```
auxflow classify

$ auxflow classify main.tex
auxflow: AMPLIFYING CYCLE -- this document may not
  converge.

  A cross-reference value influences how much content the
    document generates,
  which in turn changes that value.

  cycle (3 facts):
    page reference to 'end:generated'  main.tex:5:32
      → amount of content the document generates
        main.tex:8:3  its value changes how much content
        is generated
      → page of label 'end:generated'  main.tex:10:29
        pagination sets page of label end:generated
      → page reference to 'end:generated'  main.tex:5:32

  likely cause:
    the page reference to 'end:generated' at
      main.tex:5:32: its value changes how much content
      is generated.

...
```

Fig. 5. Document `b08`. Left: its ARG. The edge from the page reference to CONTENT_VOLUME is added by the taint analysis. Right: the report of `auxflow classify`, cut at 21 lines. Both come from the same source location data.

and the verifier stops at run six with CAP_EXCEEDED. Figure 6 shows this run and the report for an UNKNOWN document.

V. Evaluation

A. Questions and design

We ask four questions.

- RQ1 Does the classifier separate the modeled mechanisms, and which mechanisms does it miss?
- RQ2 Is the bound N at least the measured run count N^* for every certified document?
- RQ3 How do the stopping tests differ in verdict and in engine runs?
- RQ4 What does the analysis cost?

Figure 7 shows the setup. The corpus has 15 documents. Each document isolates one mechanism. We wrote the expected classification of each document before we wrote the analyzer. We included three documents that the analyzer was expected to misjudge, so that a predicted miss counts as a result and not as a defect.

The baselines are: A, a fixed schedule of three runs; B, the kernel rerun warning; C, comparison of the state after consecutive runs (the test of `latexmk`); C′, comparison of the engine inputs (`latexrun`); C″, comparison of files that were read and then rewritten in one run (Tectonic); D, AuxFlow static analysis alone; and E, the AuxFlow plan with runtime verification. Every dynamic baseline stops at eight runs. For E, an UNKNOWN document is refused, as under `auxflow run --strict`.

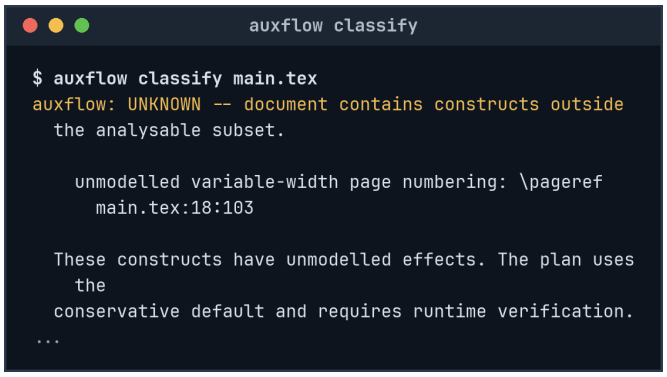
Ground truth. We run the engine eight times without a stopping rule. N^* is the smallest $k < 8$ such that the auxiliary state and the text layer from every run k to 8 equal those of run 8. This definition needs the "every run" clause. Without it, a document that alternates between two states matches at every even k and gets a false finite N^*. We found this problem in the first measurement and corrected the harness. A document without such a k is *divergent* or *oscillating*.

Environment. One container per repetition, one CPU, no network, and fixed `SOURCE_DATE_EPOCH`. The image has TeX Live 2022 from Debian bookworm, latexmk 4.79, latexrun, Tectonic 0.15.0, and Typst 0.11.0. Its digest begins with `93feb7fb6c24`. We ran ten repetitions for each document and baseline. All repetitions agree for every cell of Table III, except where the text says otherwise. We report medians, and we report no corpus-wide mean.

B. Classification and the bound (RQ1, RQ2)

Table II shows the result. Document `b08` is amplifying. Document `b05`, the table of contents, is certified and is not amplifying. These two checks guard against a vacuous analysis, and both pass. Document `b14` is amplifying too. It is a true positive that comes from over-approximation. Nothing in `b14` changes the quantity of content, only its size. Typst compiles the ported document without an error, which shows that the verdict is a conservative one.

All 7 certified documents satisfy $N \geq N^*$. The bound is exact for 4 documents and larger by one for 3. It is never



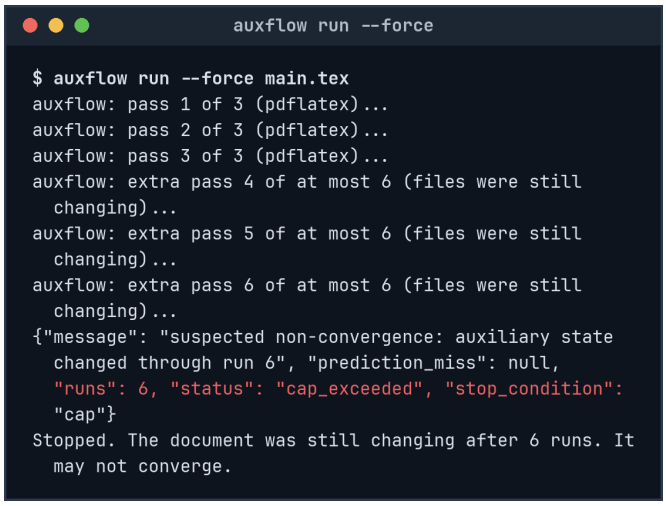


Fig. 6. Left: `auxflow classify` on `b11`. Right: `auxflow run --force` on `b08`. The run stops at the cap of six runs.

TABLE II
CORPUS. N^* IS THE MEASURED RUN COUNT (DIV.: DIVERGENT, OSC.: OSCILLATING). "BEFORE" IS THE CLASSIFICATION IN THE FIRST RUN. "NOW" IS THE CLASSIFICATION OF THE CURRENT ANALYZER. BOLD MARKS A CHANGE. N IS THE PREDICTED RUN COUNT.

ID	Mechanism	N^*	Before	Now	N
b01	no auxiliary facts	1	CERT	CERT	1
b02	backward `\ref`	2	CERT	CERT	2
b03	forward `\ref`	2	CERT	CERT	2
b04	TOC, then `\pageref`	2	CERT	CERT	3
b05	table of contents	3	CERT	CERT	3
b06	TOC, BibTeX, `\pageref`	3	CERT	CERT	4
b07	M7 footnote feedback	2	UNK	UNK	–
b08	M1 page-bounded loop	div.	AMPL	AMPL	–
b09	multi-chapter thesis	3	CERT	CERT	4
b10	M8 random auxiliary record	div.	UNK	UNK	–
b11	M3 numeral width	osc.	CERT	**UNK**	–
b12	M4 float, Roman numerals	osc.	CERT	**UNK**	–
b13	M6 `varioref` text	osc.	CERT	**UNK**	–
b14	M2 page-conditioned size	osc.	AMPL	AMPL	–
b15	M5 `ibid` citation style	3	UNK	UNK	–

TABLE III
ENGINE RUNS UNTIL A VERDICT (MEDIAN OF TEN). †: THE BASELINE ACCEPTED THE DOCUMENT AS CONVERGED ALTHOUGH THE OUTPUT WAS STALE OR THE DOCUMENT NEVER SETTLES. CAP: THE BASELINE STOPPED AT ITS LIMIT. REF.: AUXFLOW REFUSED TO RUN.

ID	N^*	A	B	C	C′	C″	E
b01	1	3	1	2	2	2	2
b02	2	3	2	2	2	2	2
b03	2	3	2	2	2	2	2
b04	2	3	2	2	2	2	2
b05	3	3	1†	3	3	3	3
b06	3	3	3	3	3	3	3
b07	2	3	2	2	2	2	ref.
b08	div.	3†	cap	cap	cap	cap	ref.
b09	3	3	3	3	3	3	3
b10	div.	3†	1†	cap	cap	cap	ref.
b11	osc.	3†	cap	cap	cap	cap	ref.
b12	osc.	3†	cap	cap	cap	cap	ref.
b13	osc.	3†	cap	cap	cap	cap	ref.
b14	osc.	3†	cap	cap	cap	cap	ref.
b15	3	3	1†	3	3	3	ref.

too small (0 cases). A larger bound is expected, because the analysis does not claim a minimum.

The analyzer flags all 6 non-convergent documents. It labels two amplifying and four UNKNOWN. It certifies none of them. This result differs from the first measurement, and Section VI explains why.

C. Stopping tests (RQ3)

Table III and Fig. 8 give the engine runs until each test reached a verdict. Table IV counts the verdicts.

Four results stand out.

First, the fixed schedule A is silently wrong on every non-convergent document. It reports three runs as done and has no way to know whether three runs were enough.

Second, the kernel warning B is unsound in both measured ways. On `b10` it accepts a document that never settles, because a changing random value produces no reference warning. On `b05` and `b15` it stops after one run, although N^* is three. The first of these is the case that the `rerunfilecheck` documentation predicts [23].

Third, the three state-comparison tests C, C′, and C″

TABLE IV
VERDICTS OVER THE CORPUS. COLUMNS A TO E ARE BASELINES. COLUMN D IS THE STATIC CLASSIFICATION ALONE.

	A	B	C	C′	C″	E	D
Correct stop, of 9 convergent documents	9	7	9	9	9	7	7
Premature stop (stale output accepted)	0	2	0	0	0	0	0
Withheld by static refusal	0	0	0	0	0	2	2
Non-convergence flagged, of 6	0	5	6	6	6	6	6
Non-convergence accepted as converged	6	1	0	0	0	0	0
Engine runs spent on the 9 convergent documents	27	17	22	22	22	17	0

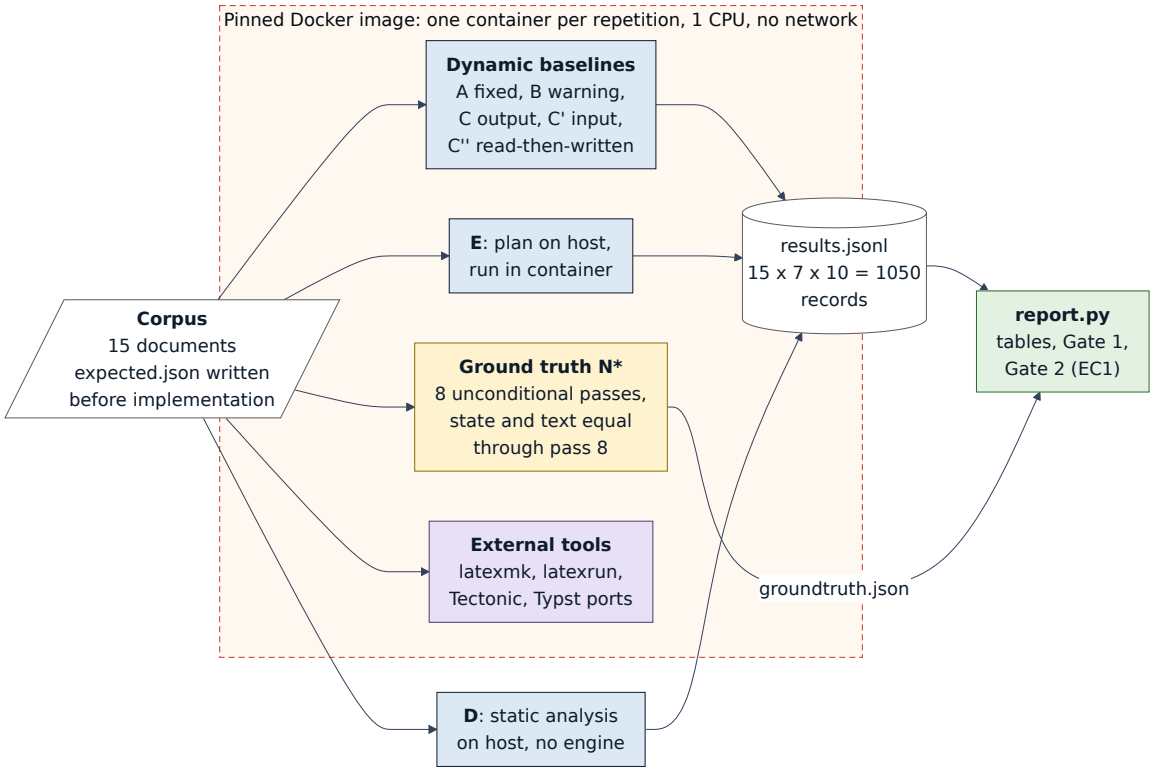


Fig. 7. Evaluation design. Baselines A to E run inside a pinned container. Baseline D runs on the host and never starts an engine.

give the same verdict on every document. They spend 17 engine runs each on the seven certified documents, and so does E. We did not reproduce a one-run gap between output comparison and input comparison on this corpus. We expected one from the manuals. We did not investigate the cause, and a larger corpus may show it.

Fourth, E equals the best dynamic test in runs on the seven documents that it executes. It spends 17 runs, against 21 for the fixed schedule. It refuses eight documents before the first run: six non-convergent ones, and the two convergent ones that contain opaque constructs (`b07` and `b15`). This is a saving of four runs over A on tiny documents, and the corpus cannot support a stronger claim. Document `b01` needs two runs under E although $N = 1$. pdfTeX writes a page-count record to every `.aux` file, so the state is never empty, and the verifier needs a second run to see two equal states. It logs a prediction miss on all ten repetitions. The prediction is right, since $N^* = 1$. The miss comes from the confirming run that any state-comparison test needs.

D. External tools (RQ3)

We ran `latexmk`, `latexrun`, and Tectonic on the same corpus. This is a cross-check and not a baseline. Every document that is not listed in Table V converges under `latexmk` and `latexrun`. Both tools fail on all six non-convergent documents. This confirms that the corpus members are real non-convergence and not artifacts of the corpus. `latexrun` also fails on `b15`, which `latexmk` accepts. Tectonic compiles only `b01` and `b08` offline, because the other documents need files from its network bundle, and we block the network. On `b08`, Tectonic prints that a rerun seems needed and stops at six passes. Typst rejects our hand port of `b08` with "loop seems to be infinite". This is a second independent implementation that reaches the same verdict as AuxFlow.

TABLE V
CONVERGENCE REPORTED BY EXTERNAL TOOLS ON THE DOCUMENTS WHERE AT LEAST ONE TOOL FAILS. N/A: TECTONIC CANNOT COMPILE THE DOCUMENT OFFLINE.

ID	latexmk	latexrun	Tectonic
b08	no	no	cap
b10	no	no	n/a
b11	no	no	n/a
b12	no	no	n/a
b13	no	no	n/a
b14	no	no	n/a
b15	yes	no	n/a

E. Cost (RQ4)

Static analysis takes 1.3 ms at the median and 10.4 ms at most over all 15 documents. The largest graph is `b09`, with 91 facts and 73 edges, at 7.5 ms. We report this time as overhead and do not subtract it from any run time. We did

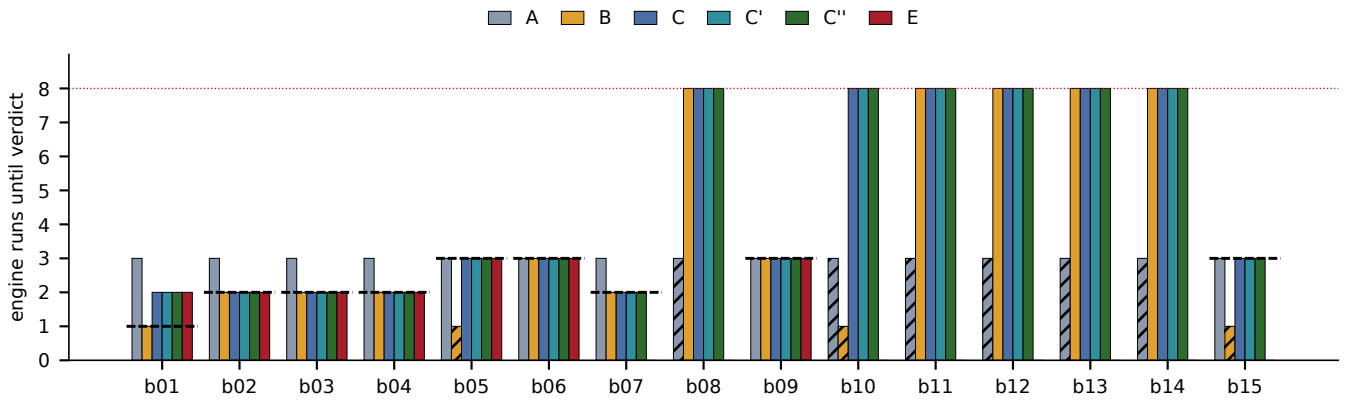


Fig. 8. Engine runs until a verdict, by document and baseline. Dashed bars mark N^*. Hatched bars mark a wrong acceptance. Missing bars for E mark a refusal. The dotted line is the cap of eight runs.

not measure engine time per run separately, so we make no claim about the ratio of analysis time to run time.

VI. A Change After the First Measurement

The first measurement ran before the guard in this section existed. Then, documents `b11`, `b12`, and `b13` were certified. That was the expected false negative. Their feedback runs through the typeset width of a reference and not through a loop, so no seed reaches them. Table II shows the old and the new classification.

We then added three syntactic guards to the lowering step. A page reference under `\pagenumbering {roman}`, Roman, alph, or Alph numbering makes the document UNKNOWN, because the numeral width depends on its value. The commands `\vref` and `\vpageref` make a document UNKNOWN for the same reason. A page reference under arabic numbering is not affected, so `b03` and `b09` stay certified. We also added a list of inert packages and commands, so that ordinary front matter does not turn a document into UNKNOWN.

This change has three consequences for how to read our results.

- The three expected misses are no longer misses. The pre-registered expectation (certified) fails on these documents, and we report the gate that checks it as failed.
- The change is post hoc. We made it after we saw the measurement, so the corpus no longer tests the guard independently.
- The guard matches syntax and not the mechanism. We did not test width feedback under arabic numbering, and we expect the analysis to certify such a document.

The analysis is still incomplete, and the incompleteness is the main limit of the method.

VII. Threats to Validity and Limitations

Corpus size. The corpus has fifteen documents, and most are synthetic. Only `b09` resembles a real document. Every statement about run counts applies to this corpus.

Author bias. The same person wrote the corpus, the expectations, and the analyzer. The expectations came first, and they are in the repository. This reduces the risk but does not remove it.

Assumption A2. The model assumes that resolving a reference does not change pagination. That assumption is false. Replacing `??` with a number can change a line break. Without the assumption, every document with a reference is one component. With it, the analysis is unsound for the extent-driven class. We chose the second option. We measure the unsoundness and do not claim to remove it.

Divergence and oscillation. Our amplifying label covers two different failures. A monotone loop violates the finite-height condition. A page-conditioned size alternates, which violates monotonicity. A split of the label would follow the theory, but the second class is mostly invisible to the analysis.

Benign cycles. The bound for a benign cycle is a heuristic, and the corpus has no benign cyclic member. We treat it as unvalidated.

Platform. The container holds TeX Live 2022. The host in our screenshots holds a newer release. Run counts may differ with other releases. We found one measurement error of this kind in our own process: leftover auxiliary files in a corpus directory made three baselines stop early on `b05`. We removed those records and ran them again.

Scope. AuxFlow analyzes a declared subset. It does not implement TeX, does not expand macros, and does not claim a minimum run count. A cycle in the ARG is not sufficient for non-convergence, and its absence is not sufficient for convergence.

VIII. Conclusion

A document can change the graph that decides its own run count. For that reason, classical pass numbering does not apply to every LaTeX document. AuxFlow tests whether it applies. On our corpus, the test flagged every

non-convergent document before the first engine run and bounded every certified document correctly. It also ran in about one millisecond.

The evidence has clear limits. The corpus is small. The width guard is syntactic and was added after the first measurement. The analysis cannot see most extent-driven feedback. The next steps are a larger corpus of real documents, a test for width feedback that does not depend on syntax, and a split of the amplifying label into divergent and oscillating cases.

Availability

The source, corpus, harness, and result files are in the AuxFlow repository [25]. Run `python paper/make_results.py` to regenerate every table in this paper from the result files.

References

[1] J. Collins, "latexmk: automatic LaTeX document generation," CTAN, https://ctan.org/pkg/latexmk, version 4.88 manual and source.

[2] Tectonic Typesetting Project, "Tectonic: a modernized, complete, self-contained TeX/LaTeX engine," https://github.com/tectonic-typesetting/tectonic, version 0.15.0.

[3] Typst contributors, "Typst: a markup-based typesetting system," https://github.com/typst/typst, version 0.11.0.

[4] Gaspar, "Does not suffice to run LaTeX a finite number of times to get cross-references right," *TUGboat*, vol. 35, no. 1, p. 16, 2014.

[5] G. V. Bochmann, "Semantic evaluation from left to right," *Communications of the ACM*, vol. 19, no. 2, pp. 55–62, 1976.

[6] H. Alblas, "A characterization of attribute evaluation in passes," *Acta Informatica*, vol. 16, no. 4, pp. 427–464, 1981.

[7] M. S. Hecht and J. D. Ullman, "A simple algorithm for global data flow analysis problems," *SIAM Journal on Computing*, vol. 4, no. 4, pp. 519–532, 1975.

[8] L. G. Jones, "Efficient evaluation of circular attribute grammars," *ACM Transactions on Programming Languages and Systems*, vol. 12, no. 3, pp. 429–462, 1990.

[9] E. Magnusson and G. Hedin, "Circular reference attributed grammars: their evaluation and applications," *Science of Computer Programming*, vol. 68, no. 1, pp. 21–37, 2007.

[10] A. Clements, "latexrun: a LaTeX wrapper that fixes the bad parts of LaTeX," https://github.com/aclements/latexrun.

[11] K.-J. Räihä and E. Ukkonen, "Minimizing the number of evaluation passes for attribute grammars," *SIAM Journal on Computing*, vol. 10, no. 4, pp. 772–786, 1981.

[12] R. Farrow, "Automatic generation of fixed-point-finding evaluators for circular, but well-defined, attribute grammars," *SIGPLAN Notices*, vol. 21, no. 7, pp. 85–98, 1986.

[13] L. Krishnan and E. V. Wyk, "Termination analysis for higher-order attribute grammars," in *Software Language Engineering (SLE 2012)*, ser. LNCS, vol. 7745, 2012, pp. 44–63.

[14] H. H. Vogt, S. D. Swierstra, and M. F. Kuiper, "Higher order attribute grammars," in *Proc. ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 1989.

[15] E. Söderberg and G. Hedin, "Circular higher-order reference attribute grammars," in *Software Language Engineering (SLE 2013)*, ser. LNCS, vol. 8225, 2013, pp. 302–321.

[16] J. B. Kam and J. D. Ullman, "Global data flow analysis and iterative algorithms," *Journal of the ACM*, vol. 23, no. 1, pp. 158–171, 1976.

[17] E. A. Lee and D. G. Messerschmitt, "Static scheduling of synchronous data flow programs for digital signal processing," *IEEE Transactions on Computers*, vol. C-36, no. 1, pp. 24–35, 1987.

[18] A. Mokhov, N. Mitchell, and S. Peyton Jones, "Build systems à la carte," *Proceedings of the ACM on Programming Languages*, vol. 2, no. ICFP, 2018.

[19] S. Erdweg, M. Lichter, and M. Weiel, "A sound and optimal incremental build system with dynamic dependencies," in *Proc. ACM International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, 2015.

[20] S. Erdweg and K. Ostermann, "Featherweight TeX and parser correctness," in *Software Language Engineering (SLE 2010)*, 2010.

[21] L. Mädje, "A programmable markup language for typesetting," Master's thesis, Technische Universität Berlin, Sep. 2022.

[22] R. Tarjan, "Depth-first search and linear graph algorithms," *SIAM Journal on Computing*, vol. 1, no. 2, pp. 146–160, 1972.

[23] H. Oberdiek, "The rerunfilecheck package: checksum based rerun checks on auxiliary files," CTAN, https://ctan.org/pkg/rerunfilecheck.

[24] CTAN contributors, "The labelschanged package," CTAN, https://ctan.org/pkg/labelschanged.

[25] U. Mazumder, "AuxFlow: static analysis for multi-pass document compilation," https://github.com/upayanmazumder/auxflow, release 0.3.0; PyPI package `auxflow`.